



Professional Documentation for

yugipedia_card_details_scraper.py

Overview

yugipedia_card_details_scraper.py is a production-grade Python script designed to scrape comprehensive Yu-Gi-Oh! card details from Yugipedia (<https://yugipedia.com>) via its web interface. The script employs concurrent processing with rate limiting to efficiently retrieve detailed information for over 13,000 cards while fully respecting Wikimedia Foundation policies and Cloudflare protection mechanisms.

Purpose

To extract complete card information including:

- Basic metadata (ID, name, type, properties)
- Monster-specific attributes (ATK, DEF, level, rank, link markers)
- Card effects and descriptions
- Archetype associations
- Complete printing history across all card sets with rarity information

Technical Requirements

Dependencies

```
pip install cloudscraper beautifulsoup4
```

- **Python:** 3.8 or higher
- **cloudscraper:** Bypasses Cloudflare JavaScript challenges
- **beautifulsoup4:** HTML parsing and data extraction
- **lxml:** (Optional) Faster HTML parsing backend

Input File Format

Required: `yugipedia_passcode_list.json`

Structure:

```
[
  {
    "name": "Blue-Eyes White Dragon",
    "card_type": "Monster Card",
    "password": "89631139",
    "url": "https://yugipedia.com/wiki/Blue-Eyes_White_Dragon"
  }
]
```

Output Files

1. `yugipedia_all_cards.json`: Successfully scraped cards with complete details
2. `yugipedia_rejected_cards.json`: Failed cards with rejection reasons and timestamps

Architecture and Design

Concurrent Processing Model

The script uses **ThreadPoolExecutor** with the following characteristics:

- **Workers:** 8 concurrent threads
- **Rate Limit:** 3 requests per second (global, thread-safe)
- **Approach:** Token bucket algorithm for precise rate limiting
- **Scraper Distribution:** Round-robin assignment of CloudScraper instances to workers

Thread-Safe Rate Limiting

```
class RateLimiter:
    """Ensures no more than 3 req/sec across all threads"""
```

Uses mutex locks to coordinate request timing globally, preventing any thread from exceeding the rate limit.

Cloudflare Bypass

Employs `cloudscraper` library which:

- Automatically solves JavaScript challenges
- Handles cookie management
- Mimics legitimate browser behavior

- Maintains User-Agent header for Wikimedia compliance

Key Features

1. Card Type Handlers

Three specialized parsing functions:

- `parse_monster_card()`: Handles Monster Cards with all attributes
- `parse_spell_card()`: Processes Spell Cards with properties
- `parse_trap_card()`: Processes Trap Cards with properties

2. Data Extraction Capabilities

Monster Cards:

- Toplevel parsing (Aqua, Beast, Dragon, etc.)
- Mechanic detection (Normal, Fusion, Synchro, Xyz, Pendulum, Link)
- Level/Rank extraction
- ATK/DEF with string support ("?", "??")
- Link markers (only actual markers, not all 8 positions)
- Pendulum scale for Pendulum monsters
- Summoning conditions for Extra Deck monsters

Spell/Trap Cards:

- Property extraction (Normal, Continuous, Quick-Play, etc.)
- Effect descriptions
- Archetype associations

All Cards:

- Complete card set history with dates, codes, and rarities
- Rarity code mapping (including 10000 Secret Rare → 10000ScR)
- Password verification against page content
- Archetype/series detection

3. Robust Error Handling

- Retry logic (3 attempts per request with exponential backoff)
- Password mismatch detection
- Network error recovery
- HTTP error handling (403, 404, 429, 500)

- Graceful degradation for missing data fields

Configuration Options

Performance Tuning

```
MAX_WORKERS = 8           # Concurrent threads (safe: 5-10)
REQUESTS_PER_SECOND = 3   # Rate limit (safe: 2-4)
```

Performance Matrix:

Workers	Req/Sec	Time (13,663 cards)	Risk Level
5	2	~114 min	None
8	3	~76 min	Low (Recommended)
10	4	~57 min	Medium
15	5	~45 min	High

User-Agent Configuration

```
USER_AGENT = 'YugipediaCardBot/1.0 (https://github.com/egera24; egera24@gmail.com) request'
```

Components:

- Bot name and version
- Contact information (GitHub + email)
- Library identification

Wikimedia Foundation Policy Compliance

User-Agent Policy

Reference: https://foundation.wikimedia.org/wiki/Policy:Wikimedia_Foundation_User-Agent_Policy

Compliance Measures:

1. ✓ Descriptive User-Agent with contact information
2. ✓ Includes "bot" keyword for traffic classification
3. ✓ Rate limiting (3 req/sec is conservative)
4. ✓ Polite request intervals
5. ✓ No browser User-Agent impersonation

6. ✓ Retry logic with backoff delays

Best Practices

- Script waits minimum 0.33 seconds between any two requests
- Uses proper API/HTML endpoints (not aggressive scraping)
- Handles errors gracefully without overwhelming server
- Can be safely used repeatedly without causing issues

Function Reference

Core Functions

`create_scraper()`

Creates CloudScraper instance configured to bypass Cloudflare.

Returns: Configured scraper object with User-Agent

`fetch_page(scraper, url, retries=3)`

Fetches HTML content from URL with retry logic and rate limiting.

Parameters:

- `scraper`: CloudScraper instance
- `url`: Target URL
- `retries`: Number of retry attempts (default: 3)

Returns: (`html_text`, `error`) tuple

`extract_atk_def(soup, is_link=False)`

Extracts ATK/DEF values, handling both integers and strings.

Special Cases:

- Regular cards: Returns integers for ATK/DEF
- Unknown values: Returns strings ("?", "??")
- Link monsters: Returns ATK and `link_rating`

`extract_link_markers(soup)`

Extracts actual link markers present on the card.

Logic: Reads text list (not images) to determine active markers

Returns: List of markers (e.g., [`"Top"`, `"Right"`, `"Bottom"`])

Card Parsing Functions

```
parse_monster_card(soup, input_card)
```

Complete parsing logic for Monster Cards.

Extracts:

- ID (password with leading zeros)
- Typeline, attribute, type
- Mechanic and effect status
- Level/Rank, ATK/DEF
- Pendulum scale (if applicable)
- Link markers (if Link monster)
- Descriptions and summoning conditions
- Archetypes and card sets

```
parse_spell_card(soup, input_card)
```

Parsing logic for Spell Cards.

Extracts:

- ID, name, type ("Spell")
- Property (Normal, Continuous, etc.)
- Description
- Archetypes and card sets

```
parse_trap_card(soup, input_card)
```

Parsing logic for Trap Cards.

Extracts:

- ID, name, type ("Trap")
- Property (Normal, Continuous, Counter)
- Description
- Archetypes and card sets

Usage Examples

Basic Execution

```
python yugipedia_card_details_scraper.py
```

Expected Console Output

```
=====
Yugipedia Card Details Scraper (BUG FIXES v2)
Wikimedia Foundation Policy Compliant
=====

User-Agent: YugipediaCardBot/1.0 (...)
Max Workers: 8 concurrent threads
Rate Limit: 3 requests/second

+++[INFO]+++ Loading cards from yugipedia_passcode_list.json...
+++[INFO]+++ Loaded 13663 cards
+++[INFO]+++ Starting concurrent card processing...

[1/13663  0.0%] ✓ Parallel Teleport          | Rate: 3.0/s | ETA: 75.8m
[2/13663  0.0%] ✓ Labrynth Cooclock         | Rate: 3.0/s | ETA: 75.7m
...
```

Output JSON Structure Example

```
{
  "id": "89631139",
  "name": "Blue-Eyes White Dragon",
  "typeline": ["Dragon", "Normal"],
  "attribute": "LIGHT",
  "type": "Dragon",
  "mechanic": "Normal",
  "effect": "no",
  "level": 8,
  "atk": 3000,
  "def": 2500,
  "description": "This legendary dragon is a powerful engine of destruction...",
  "card_sets": [
    {
      "set_name": "Legend of Blue Eyes White Dragon",
      "set_code": "LOB-001",
      "set_rarity": "Ultra Rare",
      "set_rarity_code": "UR",
      "set_release_date": "2002-03-08"
    }
  ]
}
```

Bug Fixes History

Version 2 (Current)

Bug Fix #1: ATK/DEF String Values

- **Issue:** Crash on cards with non-numeric ATK/DEF ("?", "??")
- **Solution:** Conditional integer conversion with string fallback
- **Example:** "atk": "?" instead of crash

Bug Fix #2: 10000 Secret Rare Mapping

- **Issue:** New rarity mapped to "New"
- **Solution:** Added '10000 Secret Rare': '10000ScR'
- **Result:** Proper rarity code assignment

Bug Fix #3: Link Markers Extraction

- **Issue:** All 8 markers returned instead of actual markers
- **Solution:** Read text list instead of image tags
- **Result:** Only present markers extracted

Version 1

Initial Issues Resolved:

- Cloudflare 403 blocking (solved with cloudscraper)
- Sequential processing (solved with ThreadPoolExecutor)
- ID format losing leading zeros (solved by keeping as string)

Performance Characteristics

Timing Analysis

For 13,663 cards:

- **Network latency:** ~300-500ms per request (unavoidable)
- **Cloudflare processing:** ~100-200ms per request
- **HTML parsing:** ~50-100ms per card
- **Total per card:** ~450-800ms

With 8 workers at 3 req/sec:

- Theoretical max: 10,800 cards/hour
- Actual: ~7,000 cards/hour (accounting for retries)

- **Total time: ~76 minutes**

Bottleneck Distribution

- 70% Network/Cloudflare (unavoidable)
- 20% Rate limiting (policy constraint)
- 10% Parsing/processing

Error Handling and Recovery

Rejection Scenarios

Cards are rejected and logged when:

1. Password mismatch between input and page
2. HTTP errors (404, 403, 500)
3. Network timeouts
4. Parsing exceptions
5. Invalid card type

Rejected Cards Output

```
{
  "timestamp": "2025-10-24T02:55:00",
  "total_rejected": 12,
  "rejected_cards": [
    {
      "name": "Card Name",
      "password": "12345678",
      "url": "https://yugipedia.com/wiki/Card_Name",
      "rejection_reason": "HTTP Error 404",
      "rejection_timestamp": "2025-10-24T02:45:12"
    }
  ]
}
```

Troubleshooting

Common Issues

Issue: 403 Errors

- **Cause:** Cloudflare blocking
- **Solution:** Ensure `cloudscraper` is installed

- **Verify:** Check User-Agent is set correctly

Issue: 429 Errors (Rate Limiting)

- **Cause:** Too aggressive request rate
- **Solution:** Reduce REQUESTS_PER_SECOND to 2

Issue: Frequent Timeouts

- **Cause:** Network congestion or server issues
- **Solution:** Reduce MAX_WORKERS to 5

Issue: All Cards Showing Same Link Markers

- **Cause:** Old version with bug
- **Solution:** Use current version (v2 with bug fixes)

Best Practices

For One-Time Scraping

- Use current settings (8 workers, 3 req/sec)
- Monitor first 100 cards for issues
- Check rejected_cards.json for patterns

For Regular Updates

- Reduce to 5 workers, 2 req/sec
- Schedule during off-peak hours
- Keep User-Agent contact info current

For Development/Testing

- Use 3 workers, 1 req/sec
- Test on small subset first
- Increase rate only after verification

Contact Information

Script Author: egera24

Email: egera24@gmail.com

GitHub: <https://github.com/egera24>

For issues, suggestions, or contributions, please use the contact information above.

*This documentation covers yugipedia_card_details_scraper.py v2 with all bug fixes applied.
Last updated: October 24, 2025.*